

LIGNE 42

TP — Consommer une API temps réel en JavaScript asynchrone

Version 2 - CC BY-SA 4.0 - github.com/baduritsko/ligne42

Concepteur : Caryl Le Breton (www.cafe-it.fr)

Contexte

La Ligne 42 est une ligne de métro fictive, sur le modèle de la ligne 14 du métro parisien. Elle compte 11 stations, toutes nommées d'après des femmes ayant marqué les sciences, l'histoire ou la fiction par leur découverte, leur courage ou leur rôle. Une quinzaine de trains y circulent en permanence, dans les deux sens.

Un serveur met à votre disposition, en continu, une API qui décrit l'état de tous les trains de la ligne à un instant donné : leur position, leur vitesse, leur sens de circulation et le poids de chacun de leurs wagons (qui reflète la fréquentation).

Votre mission : construire une page web qui interroge cette API à intervalles réguliers et qui affiche, de façon claire et dynamique, ce qui se passe sur la ligne.



Objectifs pédagogiques

- Consommer une API avec fetch et async/await.
- Rafraîchir une interface à intervalles réguliers sans recharger la page.
- Manipuler le DOM dynamiquement à partir de données reçues en JSON.
- Retravailler vos bases HTML/CSS : mise en page, lisibilité, hiérarchie visuelle.

Ce TP peut aussi être vu comme un exercice complémentaire de mise en page HTML/CSS : une interface qui affiche des données changeantes pose des questions de structure et de style qu'une page statique ne pose pas (que faire quand une valeur change, quand un élément apparaît ou disparaît, etc.).

L'API

L'API est accessible à l'adresse suivante :

```
https://www.cafe-it.fr/ligne42/api-trains.php
```

Chaque appel renvoie un objet JSON contenant la liste de tous les trains actuellement en circulation sur la ligne, sous la forme suivante :

```
{
  "trains": [
    {
      "sens": "A",
      "position": 6750,
      "vitesse": 30,
      "attente": 0,
      "rames": [18500, 21300, 19800, 22100, 17600]
    },
    { ... }
  ]
}
```

Signification des champs :

Champ	Type	Signification
sens	string	"A" ou "B". A = vers Cheng Xin (distances croissantes), B = vers Hypatia (distances décroissantes).
position	int	Position en mètres de l'avant du train (au niveau du feu de circulation), depuis le début de la ligne.
vitesse	int	Vitesse actuelle en km/h (palier de 5, de 0 à 45). 0 = train à quai ou immobilisé.
attente	int	Nombre de cycles restants avant le redémarrage, si le train est à quai.
rames	array	Tableau de 5 poids en kg (un par wagon), chacun compris entre 15 000 et 25 000 kg.

La simulation des erreurs

L'API peut envoyer des erreurs à intercepter dans le script JS :

- erreur=delai : l'API répond très lentement
- erreur=format : l'API envoie des données avec un format incorrect
- erreur=incomplet : l'API envoie des données JSON incomplète (données tronquées)

Exemple : <https://www.cafe-it.fr/ligne42/api-trains.php?erreur=delai>

Les stations

La distance indiquée correspond au milieu du quai. La position d'un train que vous recevrez via l'API correspond, elle, à l'avant du train — il y a donc un écart de 50 mètres entre la position d'un train à l'arrêt et la distance affichée ci-dessous pour sa station.

Station	Distance au milieu du quai	Plus d'informations
---------	----------------------------	---------------------

Station	Distance au milieu du quai	Plus d'informations
Hypatia	60	Philosophe, astronome et mathématicienne grecque
Trotula de Salerne	997	Femme médecin du Moyen Âge
Ada Lovelace	2311	Pionnière de la science informatique
Marie Curie	3942	Scientifique et double Prix Nobel
Simone Veil	5150	Femme politique française
Wangari Maathai	6700	Militante écologiste, biologiste et professeure kényane
Katherine Johnson	8599	Ingénieure spatiale, mathématicienne et physicienne
Valentina Terechkova	9840	Femme politique et cosmonaute
Mae Jemison	11001	Ingénieure, astronaute et médecin
Paula Myo	12486	Personnage de fiction
Cheng Xin	13940	Personnage de fiction

Rythme de rafraîchissement

Les données de l'API évoluent toutes les 4 secondes. Il n'est pas utile d'interroger le serveur plus souvent que cela. Pensez à choisir un intervalle de rafraîchissement cohérent dans votre code (au moins 4 secondes).

Pistes de réalisation

Il n'y a pas une seule bonne façon de représenter ces données. Voici deux pistes pour vous inspirer — vous êtes libres de vous en éloigner, de les combiner, ou d'imaginer autre chose entièrement.

Piste 1 — Le panneau de quai

Dans la plupart des stations de métro, un panneau lumineux annonce les prochains trains à venir, leur temps d'attente estimé, et parfois leur fréquentation (codes couleur vert / jaune / rouge selon le remplissage des wagons).

Une variante de ce principe peut afficher, pour une station donnée et un sens donné :

- le temps d'attente avant les deux prochains trains,
- un indicateur visuel quand un train approche,
- la fréquentation de chaque wagon du train à quai ou en approche, sous forme de couleur ou de jauge.

C'est un exercice intéressant de calcul (estimer un temps d'arrivée à partir d'une position et d'une vitesse) autant que d'affichage.

Panneau d'affichage à la station *Simone Veil* dans le sens A (terminus à Cheng Xin)



Démo : <https://www.cafe-it.fr/ligne42/panneau/>

Piste 2 — La vue centre de régulation

À l'opposé du panneau de quai, qui se concentre sur une station, on peut imaginer une vue d'ensemble de la ligne, comme celle qu'utiliserait un centre de régulation pour superviser le trafic en temps réel.

Cette vue pourrait représenter :

- la ligne dans son ensemble, avec les stations positionnées,
- chaque train placé à sa position réelle, avec son sens de circulation,
- un indicateur de vitesse ou d'arrêt par train,
- éventuellement un niveau de détail au clic ou au survol d'un train (fréquentation, numéro, etc.).

Cette piste pose davantage de questions de positionnement CSS (placer des éléments proportionnellement à une distance) que la précédente.

Démo : à venir

Autres pistes possibles

- Un tableau de bord textuel, simple et efficace, sans représentation graphique de la ligne.
- Une vue centrée sur un seul train (sa fiche d'identité, mise à jour en direct).
- Un indicateur global de fréquentation de la ligne, station par station.

Toute proposition cohérente et fonctionnelle est valable. L'important est la justesse de l'utilisation de l'API et la clarté de ce qui est montré, pas la conformité à un modèle précis.

Consignes et déroulement

Ce TP se déroule sur plusieurs séances courtes. Vous travaillez seul(e), à votre rythme, en construisant progressivement votre interface.

1. Prenez le temps d'observer les données brutes renvoyées par l'API avant d'écrire la moindre ligne d'interface : ouvrez l'URL dans votre navigateur, ou affichez le JSON reçu dans la console.
2. Construisez d'abord une version minimale qui affiche, même de façon brute, les données reçues.
3. Mettez en place le rafraîchissement automatique avant de soigner la présentation.
4. Améliorez progressivement la mise en forme HTML/CSS, une fois la logique JavaScript fonctionnelle.
5. Gérez le cas où l'appel à l'API échoue (réseau coupé, serveur indisponible) : votre page ne doit pas casser silencieusement.

Livrables

- Une page HTML, un fichier CSS et un fichier JavaScript (ou plus, si vous structurez votre code en plusieurs fichiers).
- Le code doit fonctionner directement à l'ouverture du fichier HTML dans un navigateur, en interrogeant l'API en ligne.

Pour aller plus loin

Si vous avez terminé une version qui fonctionne et que vous voulez approfondir, quelques pistes :

- Calculer et afficher un temps d'arrivée estimé, en tenant compte des arrêts à quai à venir, pas seulement de la distance et de la vitesse instantanée.
- Rendre votre interface responsive (utilisable aussi bien sur un grand écran que sur un téléphone).
- Ajouter une transition ou une animation lors du changement de position d'un train, plutôt qu'un déplacement brut.
- Permettre de changer de station ou de sens directement depuis l'interface, sans modifier le code.
- Utiliser un canvas pour représenter la ligne et les trains.
- Modifier les paramètres de la requête asynchrone pour générer des erreurs dans le flux JSON.